

110612117 張仲瑜 人工智慧 HW3 report

Part 1-1 Minimax Search

```
132 # Begin your code
133 def minimax(gameState, depth, agent):
134     if gameState.isWin() or gameState.isLose() or depth == self.depth:
135         return self.evaluationFunction(gameState)
136     if agent == 0:
137         return max(minimax(gameState.getNextState(agent, action), depth, 1) for action in gameState.getLegalActions(agent))
138     else: #ghost
139         NextAgent = agent + 1
140         if NextAgent == gameState.getNumAgents():
141             NextAgent = 0
142             depth = depth + 1
143         return min(minimax(gameState.getNextState(agent, action), depth, NextAgent) for action in gameState.getLegalActions(agent))
144 legalMoves = gameState.getLegalActions(0)
145 scores = [minimax(gameState.getNextState(0, action), 0, 1) for action in legalMoves]
146 maxScore = max(scores)
147 maxIndex = [index for index in range(len(scores)) if scores[index] == maxScore]
148 TakeIndex = random.choice(maxIndex)
149 return legalMoves[TakeIndex]
150 # End your code
```

Line 134~143 :

根據 agent 順序取得 agent 與 legalActions 後進行下面邏輯：

如果遊戲結束或遞迴數達到 depth 限制，就回傳當下 evaluation Function 的結果；否則根據 agent 類型，為小精靈(0) 的話就回傳 Maximum，為小鬼的話(>0)就回傳 Minimum(選最差 = 對手選最好)。

如果該 agent 是此層最後一個 agent，結束後將 agent 設為小精靈、層數+1 作為下層開頭。

Line 144~149 :

取得小精靈可以做的 action 後，第一隻小鬼呼叫上面寫好的 minimax function，讓小精靈隨機在取得最高分數的 action 中選一個執行。

Part 1–2 Expectimax Search

```
166 def expectiMax(gameState, depth, agent):
167     if gameState.isWin() or gameState.isLose() or depth == self.depth:
168         return self.evaluationFunction(gameState)
169     if agent == 0:
170         return max(expectiMax(gameState.getNextState(agent, action), depth, 1) for action in gameState.getLegalActions(agent))
171     else: #ghost
172         NextAgent = agent + 1
173         if NextAgent == gameState.getNumAgents():
174             NextAgent = 0
175             depth = depth + 1
176         return sum(expectiMax(gameState.getNextState(agent, action), depth, NextAgent) / float(len(gameState.getLegalActions(agent)))
177                    for action in gameState.getLegalActions(agent))
178 legalMoves = gameState.getLegalActions(0)
179 scores = [expectiMax(gameState.getNextState(0, action), 0, 1) for action in legalMoves]
180 maxScore = max(scores)
181 maxIndex = [index for index in range(len(scores)) if scores[index] == maxScore]
182 TakeIndex = random.choice(maxIndex)
183
184 return legalMoves[TakeIndex]
```

整體運作跟 Minimax 差不多，不同的是在 Line176~177 處：

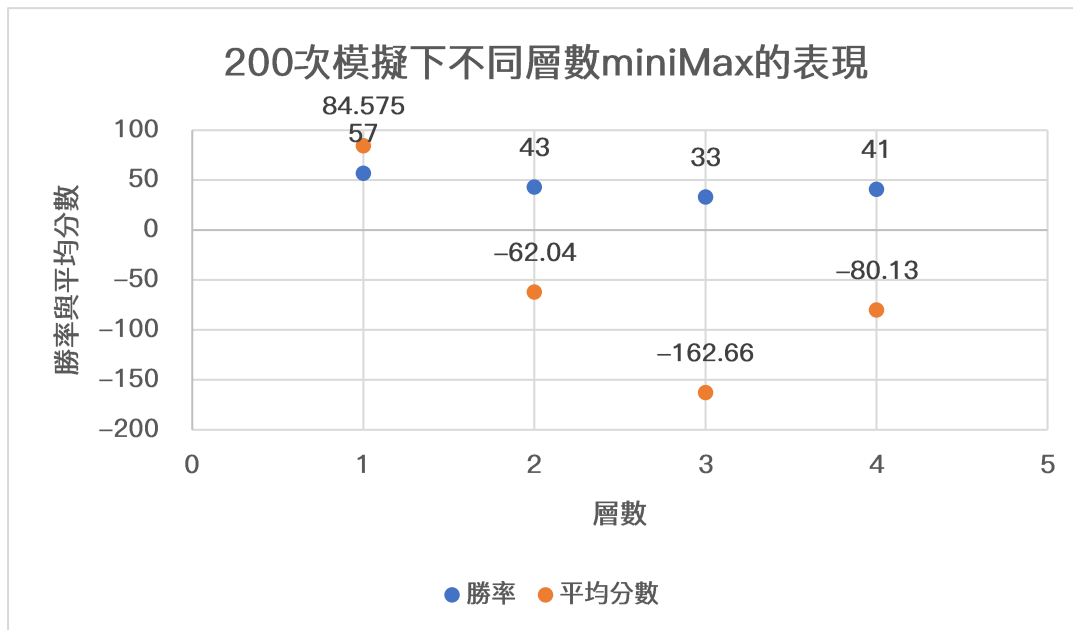
回傳小鬼所有 legalActions 得到的 expecteiMax 加總後平均來模擬小鬼的隨機決策。

Part 1 Obesrvation:

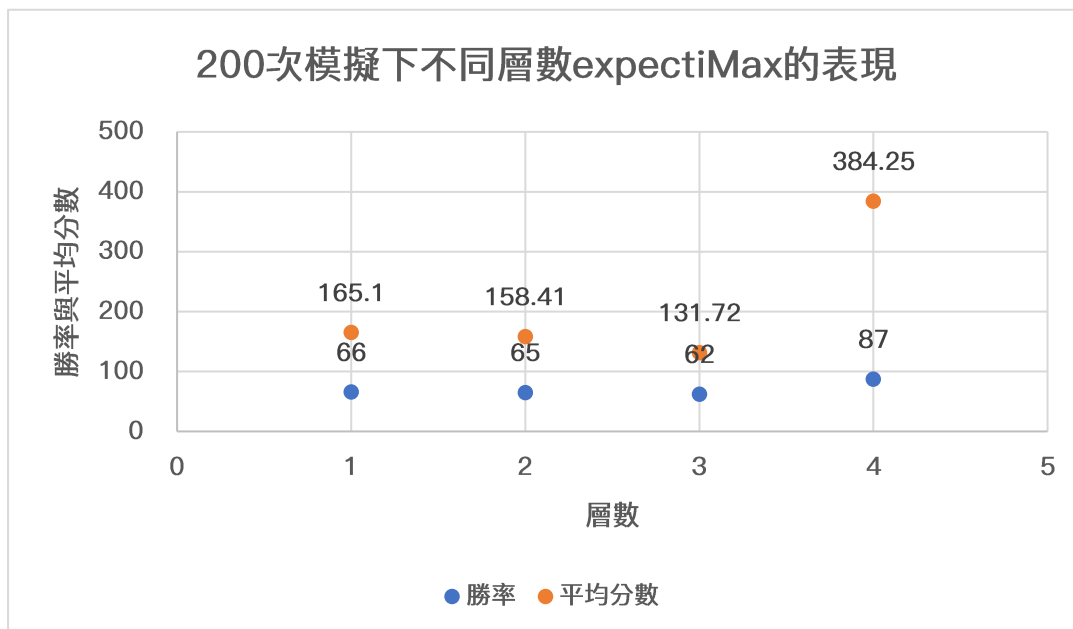
- Make sure you understand why pac man rushes to the closest ghost in this case:

```
python pacman.py -p Minimax/ ExpectimaxAgent -l trappedClassic -a
depth=3
```

在 trappedClassic 中，小精靈看似無路可逃，由於 miniMax 中總是會預想最糟情況發生(對手選最好)，在預設一定會死的情況下，降低時間上的分數損失直接自殺為 Minimax 的上策；然而在 expectiMax 中，會因為小鬼的行動隨機性(小精靈不一定會死)而讓小精靈去嘗試吃到更多小膠囊。



由上圖可以發現越高的層數不見得有越高的分數及勝率，推測是受到存活時間扣分的影響，考慮較多層可能會受到過遠的因素干擾而無法全速結束遊戲，更糟的狀況是仍然輸掉了遊戲，會損失更多分。



比起前者，ExpectiMax 明顯表現較好，勝率也高上許多，且層數的影響性也較前者更大，在更高的層數有者較好的表現

Part 2-1 Value Iteration

```
53 def runValueIteration(self):
54     # Write value iteration code here
55     """ YOUR CODE HERE """
56     # Begin your code
57     for i in range(self.iterations):
58         tmpCounter = util.Counter()
59         for state in self.mdp.getStates():
60             if self.mdp.isTerminal(state):
61                 tmpCounter[state] = 0 #no available action --> 0
62             else:
63                 maxValue = max([self.computeQValueFromValues(state, action)
64                                for action in self.mdp.getPossibleActions(state)])
65                 tmpCounter[state] = maxValue
66         self.values = tmpCounter
67     # print(self.values)
68     # End your code
```

Line 53~66:

在一次次的 iteration 中，把每個 state 的 value 更新成運用下面的 Qvalue 最大值計算涵式，回傳的該 state 下 legalActions 中 Qvalue 最大值。

Line 61~65 :

如果是終止 state 則設 value 為 0，否則就去進行 Qvalue 計算並選出最大值

```
78 def computeQValueFromValues(self, state, action):
79     """
80     Compute the Q-value of action in state from the
81     value function stored in self.values.
82     """
83     """ YOUR CODE HERE """
84     # Begin your code
85     Qvalue = 0.0
86     transitionFunctions = self.mdp.getTransitionStatesAndProbs(state, action)
87     for nextState, prob in transitionFunctions:
88         Qvalue += prob * (self.mdp.getReward(state, action, nextState) + (self.discount * self.values[nextState]))
89     return Qvalue
```

Line 78~89

透過 transitionFunction 得到 nextState 與機率後，把 state 進行所有可能

nextState 的 reward 加上 衰減係數 * nextState 的最大 reward 期望值，最後再乘上 nextState 出現的機率來得到 QValue 的值，總和即為回傳值。

```
92 def computeActionFromValues(self, state):
93
94     # Begin your code
95     maxValue = float('-inf')
96     bestAction = None
97     #check for terminal
98     if self.mdp.isTerminal(state):
99         return None
100     else:
101         for possibleAct in self.mdp.getPossibleActions(state):
102             value = self.getQValue(state, possibleAct)
103             if value > maxValue:
104                 maxValue = value
105                 bestAction = possibleAct
106         return bestAction
107     # End your code
```

Line 103~105 :

初始化最大值跟最佳 action 為負無限大與 None。

Line 106~107 :

如果為終止 state，回傳 None。

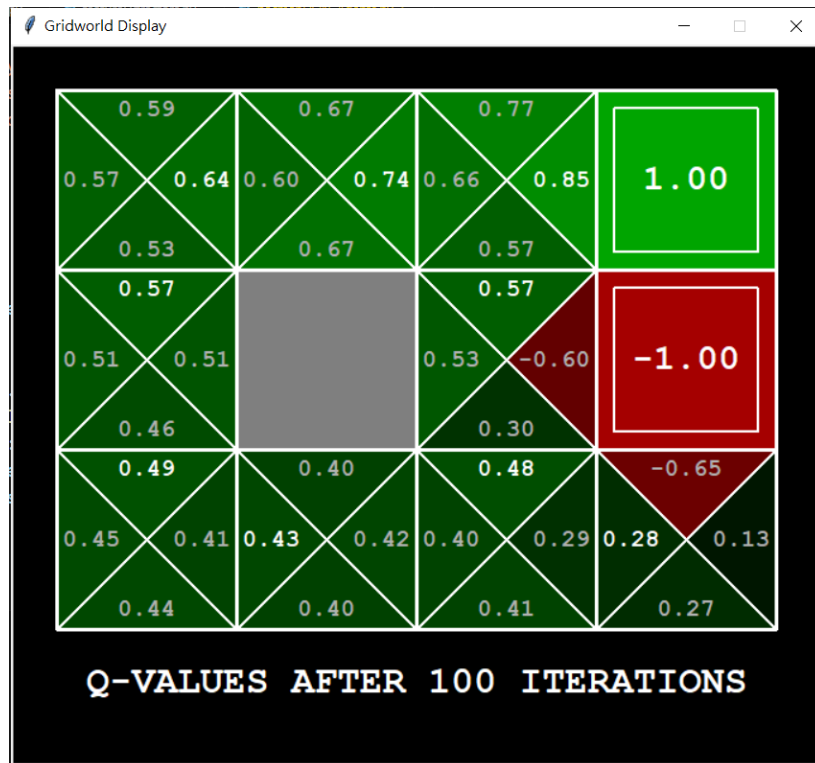
Line 108~114 :

根據最大 Qvalue 回傳對應的 bestAction。

Part 2–1 Observation

- The following command loads your `ValueIterationAgent`, which will compute a policy and execute it 10 times. Press a key to cycle through values, Q-values, and the simulation. You should find that the value of the start state ($V(\text{start})$, which you can read off of the GUI) and the empirical resulting average reward (printed after the 10 rounds of execution finish) are quite close.

```
python gridworld.py -a value -i 100 -k 10
```



EPISODE 10 COMPLETE: RETURN WAS 0.5904900000000002

AVERAGE RETURNS FROM START STATE: 0.4842278406090001

起始狀態 $v(\text{start})$ 的值與 100 輪後產生的 empirical mean 相當接近

- On the BookGrid, running value iteration for 5 iterations should give you this output:

```
python gridworld.py -a value -i 5
```



與預期輸出相同

Part 2-2 Q-learning

```
70 def computeValueFromQValues(self, state):
71     """
72     Returns max_action Q(state,action)
73     where the max is over legal actions. Note that if
74     there are no legal actions, which is the case at th
75     terminal state, you should return a value of 0.0.
76     """
77     """*** YOUR CODE HERE ***"""
78     # Begin your code
79     maxValue = float('-inf')
80     for action in self.getLegalActions(state):
81         value = self.getQValue(state, action)
82         if value > maxValue:
83             maxValue = value
84     if maxValue == float('-inf'):
85         maxValue = 0.0
86     return maxValue

91 def computeActionFromQValues(self, state):
92     """
93     Compute the best action to take in a state. Note that if there
94     are no legal actions, which is the case at the terminal state,
95     you should return None.
96     """
97     """*** YOUR CODE HERE ***"""
98     # Begin your code
99     maxValue = float('-inf')
100     bestAction = None
101     for action in self.getLegalActions(state):
102         value = self.getQValue(state, action)
103         if value > maxValue:
104             maxValue = value
105             bestAction = action
106     return bestAction
107     # util.raiseNotDefined()
108     # End your code
```

以上兩部分與 Part 2-1 邏輯相同

```
55 def getQValue(self, state, action):
56     """
57     Returns Q(state,action)
58     Should return 0.0 if we have never seen a state
59     or the Q node value otherwise
60     """
61     """*** YOUR CODE HERE ***"""
62     # Begin your code
63     if (state, action) not in self.Qvalues:
64         return 0.0
65     return self.Qvalues[(state, action)]
66     # util.raiseNotDefined()
67     # End your code
```

如果沒看過 state 與 action 的 pair，則回傳 0.0

否則回傳該 pair 的 Q-value

```
136 def update(self, state, action, nextState, reward):
137     """
138     The parent class calls this to observe a
139     state = action => nextState and reward transition.
140     You should do your Q-Value update here
141
142     NOTE: You should never call this function,
143     it will be called on your behalf
144     """
145     """ YOUR CODE HERE """
146     # Begin your code
147     if (state, action) not in self.Qvalues:
148         self.Qvalues[(state, action)] = 0.0
149     diff = reward + (self.discount * self.computeValueFromQValues(nextState)) - self.Qvalues[(state, action)]
150     self.Qvalues[(state, action)] = self.Qvalues[(state, action)] + (self.alpha * diff)
151     # util.raiseNotDefined()
152     # End your code
```

Line 147~148:

如果不存在 state 與 action 的 pair 在 self.Q-value 中，則設為 0.0

Line 149:

計算執行該 action 的 Q-value 與原本 state 的 Q-value 的差距

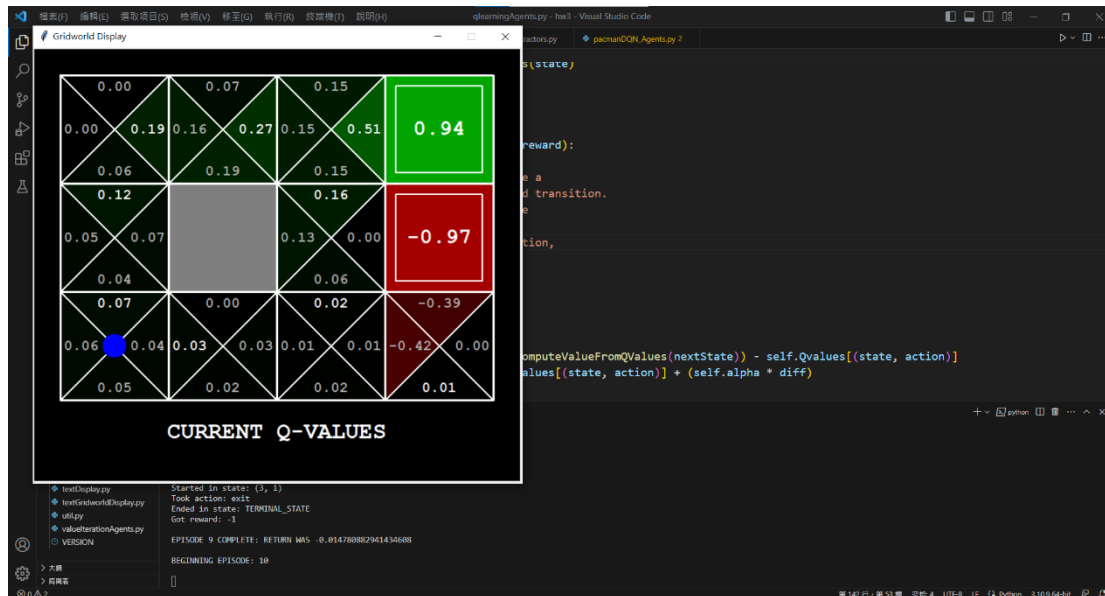
Line 150:

更新原本 state 的 Q-value 為原值+學習率*差距的版本

Part 2–2 Observation

- With the Q-learning update in place, you can watch your Q-learner learn under manual control, using the keyboard:

```
python gridworld.py -a q -k 10 -m
```



有 noise 的版本，10 個 episode 的手動操作結果，可以依步觀察，相當有趣

Part 2–3 epsilon-greedy action selection

```
110 def getAction(self, state):  
111     # Pick Action  
112     legalActions = self.getLegalActions(state)  
113     action = None  
114     """ YOUR CODE HERE """  
115     # Begin your code  
116     if len(legalActions) == 0:  
117         return None  
118     if util.flipCoin(self.epsilon):  
119         return random.choice(legalActions)  
120     else:  
121         return self.computeActionFromQValues(state)  
122     # util.raiseNotDefined()  
123     # End your code
```

Line 116~117:

如果沒有任何 legalActions，則回傳 None

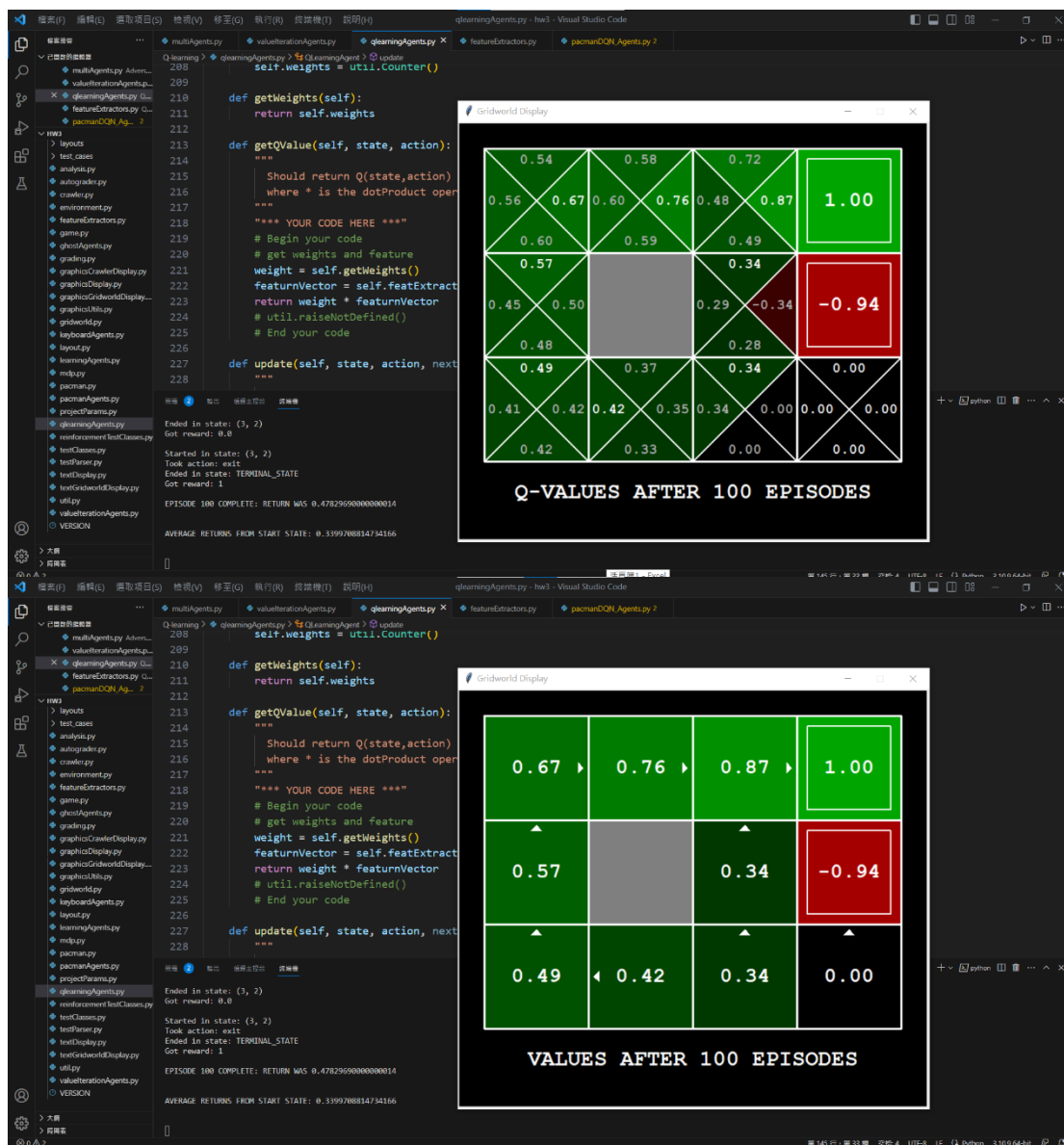
Line 118~121:

以 epsilon 機率隨機 action，否則回傳計算的最佳 action

Part 2–3 Observation

- After implementing the `getAction` method, observe the following behavior of the agent in gridworld.

```
python gridworld.py -a q -k 100
```

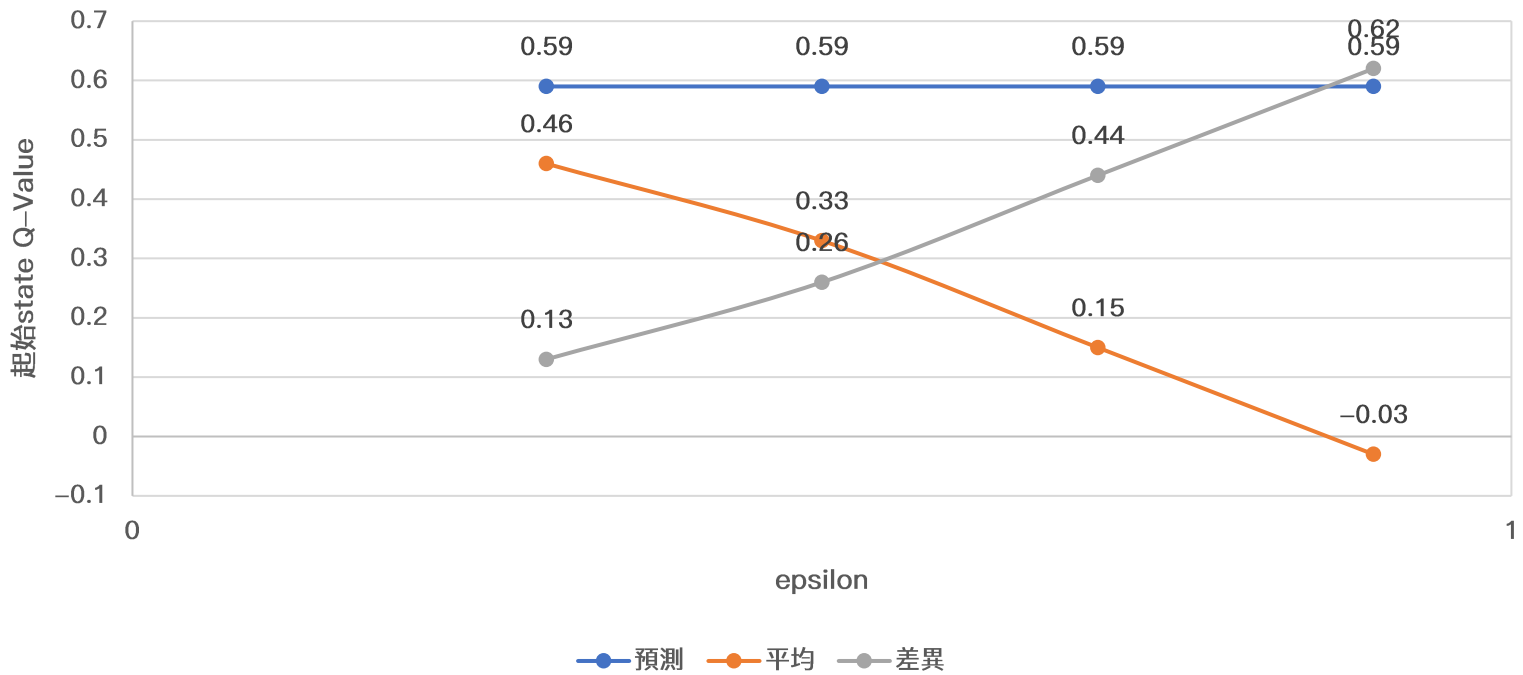


因為 epsilon 造成的隨機 action，導致 average return 低於預測的 Q-values

- You can also observe the following simulations for different epsilon values. Does that behavior of the agent match what you expect? (discuss it in your report)

```
python gridworld.py -a q -k 100 --noise 0.0 -e 0.9/0.7/0.5/0.3
```

100次訓練下不同epsilon預測與平均起始state Q-value的差異



由上圖驗證，epsilon 越小的情況下，隨機選擇 action 的機率下降，預測與實驗平均的起始狀態 Q-Value 誤差也就隨之下降。

Part 2-4 Approximate Q-learning :

```

213 def getQValue(self, state, action):
214     """
215     Should return Q(state,action) = w * featureVector
216     where * is the dotProduct operator
217     """
218     """ YOUR CODE HERE """
219     # Begin your code
220     # get weights and feature
221     weight = self.getWeights()
222     featureVector = self.featureExtractor.getFeatures(state, action)
223     return weight * featureVector
224     # util.raiseNotDefined()
225     # End your code

```

將 weight 和 feature 進行 dotProduct 後回傳

```

227 def update(self, state, action, nextState, reward):
228     """
229     Should update your weights based on transition
230     """
231     """ YOUR CODE HERE """
232     # Begin your code
233     diff = reward + (self.discount * self.computeValueFromQValues(nextState)) - self.getQValue(state, action)
234     weight = self.getWeights()
235     featureVector = self.featureExtractor.getFeatures(state, action)
236
237     for key in featureVector.keys():
238         featureVector[key] = featureVector[key] * self.alpha * diff
239         weight[key] += featureVector[key]
240     self.weights = weight.copy()
241     # End your code

```

Line 233~235 :

計算進行該 action 後 Q-value 的差異、讀取 weight 和目前 feature。

Line 237~239 :

根據 學習率與差異值與原 feature 更新對應 feature，之後再加入到對應 weight

最後更新 weights

Part 2–4 Observation

- By default, `ApproximateQAgent` uses the `IdentityExtractor`, which assigns a single feature to every (state, action) pair. With this feature extractor, your approximate Q-learning agent should work identically to `PacmanQAgent`. You can test this with the following command:

```
python pacman.py -p ApproximateQAgent -x 2000 -n 2010 -l smallGrid
```

```
Average Score: 501.4
Scores:      503.0, 503.0, 503.0, 499.0, 503.0, 503.0, 503.0, 503.0, 499.0, 495.0
Win Rate:    10/10 (1.00)
Record:      Win, Win, Win, Win, Win, Win, Win, Win, Win, Win
PS C:\Users\vince\source\hw3\Q-learning>
```

- Once you're confident that your approximate learner works correctly with the identity features, run your approximate Q-learning agent with our custom feature extractor, which can learn to win with ease:

```
python pacman.py -p ApproximateQAgent -a extractor=SimpleExtractor -x 50 -n
60 -l mediumGrid
```

```
Average Score: 528.2
Scores:      527.0, 527.0, 529.0, 529.0, 529.0, 527.0, 527.0, 529.0, 529.0, 529.0
Win Rate:    10/10 (1.00)
Record:      Win, Win, Win, Win, Win, Win, Win, Win, Win, Win
```

兩者差異不大，有關於 Extractor 的改寫，試了許多方法，包括讓小精靈去吃無效化的鬼、更激進的去吃 capsule 等，但都無明顯突破 7/10 的優勢

Part 3 DQN Observation

```
25 # model parameters
26 model_trained = False
27
28 GAMMA = 0.95 # discount factor
29 LR = 0.01    # learning rate
30
31 batch_size = 16          # memory replay batch size
32 memory_size = 50000      # memory replay size
33 start_training = 250     # start training at this episode
34 TARGET_REPLACE_ITER = 100 # update network step
35
36 epsilon_final = 0.05    # epsilon final
37 epsilon_step = 7500
```

跑到天荒地老，效果仍然不如 Pretrained 完成的版本

Pretrained 版本結果

```
Average Score: 1725.6
Scores:      1735.0, 1749.0, 1756.0, 1702.0, 1686.0
Win Rate:    5/5 (1.00)
Record:      Win, Win, Win, Win, Win
PS C:\Users\vince\source\hw3\DQN>
```

問題與討論

Q1. What is the difference between On-policy and Off-policy

	On-Policy	Off-Policy
策略差異	行為策略和目標策略相同	行為策略與目標策略不同
反應即時性	反應及時	不同策略數據傳遞需要時間
靈活度	較低，受到一個策略的限制	較高
使用樣本效率	較低，只用目前策略產生的數據	較高，可以重複使用多個策略的數據
變異性	較小，使用行為策略的學習數據	較大，數據來源有誤差

Q2. Briefly explain value-based, policy-based and Actor-Critic. Also, describe the value function $V_{\pi}(S)$.

Value-Based: 透過訓練 Critic 評價 actor- π (行為) 好壞，學習將 value function 最大化，而不是學習策略。

policy-based: 根據一策略進行優化，產生最佳的隨機策略分布，能夠找到策略將長期累積的回報最大化。

$V_{\pi}(S)$: 以 actor π 與環境中 state 互動， $V_{\pi}(S)$ 表示該 state 到遊戲結束累積

的 reward 的期望值大小。

Q3. What is the difference between Monte–Carlo (MC) based approach and Temporal–difference (TD) approach for estimating $V_{\pi}(S)$.

MC–based approach: 當 actor 每次看到 state，就去計算該 state 接下來到遊戲結束累積的 reward 為何，最後加總所有 reward 的總和，在結束時才能做出統計。由於遊戲本身具有隨機性，因此此方法變異性較大。

TD approach: 相較於前者，不需要等到整個遊戲結束，透過

$V_{\pi}(S_t) = V_{\pi}(S_{t+1}) + r$ 將下個時間點的累積 reward 加上這個時間點的 reward 即為這個時間點的累積 reward。 $V_{\pi}(S_t) - V_{\pi}(S_{t+1})$ 越接近 r 越好。隨機性在 r ，不像前者在整個遊戲，變異性較小。

Q4. Describe State–action value function $Q_{\pi}(s,a)$ and the relationship between $V_{\pi}(S)$ in Q–learning.

$Q_{\pi}(s,a)$: 使用 actor– π 與環境互動的情況下，在 State s 下採取 action a 的累積期望值會是多少； $V_{\pi}(s) = \sum [\pi(a|s) * Q_{\pi}(s, a)]$ ，state 累積 reward = actor 選擇每個 action 的機率 * 該 state 下選擇該 action a 所帶來的累積 reward 的加總。

Q5. Describe following tips Target Network, Exploration and Replay

Buffer using in Q-learning.

Target Network: 由於 Q-learning 的公式有類 TD 概念的方法在內，實際上不好訓練，output 為 $Q_{\pi}(s_t, a_t)$ ，要貼近的目標函式是會變動的 $(\text{target})r_t$

$+Q_{\pi}(s_{t+1}, \pi(s_{t+1}))$ ，透過固定其中一個 Q_{π} ，通常固定 s_{t+1} ，此被固定的模型即稱為 Target Network，產生目標給另一個模型去貼近，來簡化複雜度

Exploration: 為避免 argmax 出現獨佔現象，根據 Q-value 定義一個機率分步，對應 Q-value 越高的 action 機率就越高，但也提供了機率低的 action 一些機會。

Reply Buffer: 將紀錄 policy- π 與環境互動收集的資料存於 Reply Buffer，在訓練時就直接從 Buffer 抓資料出來更新 Q-function，可以增加 batch 時資料的多樣性(多個 π 的累積經驗)、減少與環境互動取得資料花費的時間。

Q6: Explain what is different between DQN and Q-learning.

DQN 為 Q-learning 的延伸變種，將 Q-learning 中的 Q 表以神經網路替換，以目前的 state 作為輸入，輸出所有可能 action 的 Q-value，並最小化目標 Q 值與預測 Q 值間的 loss 來最佳化。這樣可以解決 Action 與 State 數量增加時 Q 表膨脹，導致 Q-value 計算時間代價變高的問題，並使用 Replay Memory 隨機採樣，以抹除強化學習資料的順序性。

Problem meet and solve:

可以說是處處皆問題，包括無數個 state 跟 nextState 打錯浪費一堆時間，但就是沒找到、寫到最後忘記要回傳值等，但秉持著耐心學習、觀摩各種技術文章，最終還是讓自己更進步了一點，希望未來有更多時間可以更完整的吸收整套資訊。